

Module 16 – Cloud Fundamentals

1. Introduction to Cloud Computing

What is Cloud Computing?

Cloud Computing is the delivery of computing resources such as servers, storage, databases, networking, software, and analytics over the internet ("the cloud") on a pay-as-you-go basis.

Instead of purchasing and maintaining physical hardware, organizations can rent computing resources from cloud providers whenever needed.

Example

Instead of buying a physical server to host a website, a company can rent a virtual server from **Amazon Web Services (AWS)** and pay only for the resources used.

Why Cloud Computing?

Traditional IT infrastructure requires organizations to:

- Purchase expensive hardware.
- Maintain data centers.
- Upgrade servers periodically.
- Handle backups and disaster recovery.
- Manage network security.

Cloud computing eliminates most of these responsibilities by allowing cloud providers to manage the underlying infrastructure.

Traditional IT Deployment vs Cloud Computing

Traditional IT	Cloud Computing
Own physical servers	Rent virtual resources
High upfront cost	Pay only for usage
Manual scaling	Automatic scaling
Maintenance handled by organization	Managed by cloud provider
Slower deployment	Resources available in minutes
Limited flexibility	Highly flexible and scalable

Advantages of Cloud Computing

- Reduced infrastructure costs.
- On-demand resource allocation.
- High availability.
- Easy scalability.
- Global accessibility.
- Automatic updates and maintenance.

- Faster application deployment.

Challenges of Cloud Computing

- Internet dependency.
- Data privacy concerns.
- Vendor lock-in.
- Compliance requirements.
- Ongoing operational costs if resources are not managed efficiently.

Virtualization

What is Virtualization?

Virtualization is the process of creating virtual versions of physical computing resources such as servers, storage, or operating systems.

Using a technology called a **Hypervisor**, a single physical machine can run multiple independent virtual machines (VMs), each with its own operating system and applications.

Benefits of Virtualization

- Better hardware utilization.
- Lower infrastructure costs.
- Easier backup and recovery.
- Faster provisioning of servers.
- Isolation between applications.
- Improved scalability.

Service-Oriented Architecture (SOA)

What is SOA?

Service-Oriented Architecture (SOA) is a software design approach where applications are built as a collection of independent services that communicate over a network.

Each service performs a specific business function and can be reused by multiple applications.

Characteristics

- Loosely coupled services.
- Reusable components.
- Independent deployment.
- Standardized communication (e.g., REST, SOAP).

Example

An e-commerce application may have separate services for:

- User Management
- Payment Processing
- Product Catalog
- Order Management
- Notification Service

Each service operates independently while working together to deliver the complete application.

Cloud vs On-Premises

Cloud Infrastructure

Infrastructure is hosted and managed by a cloud provider such as AWS, Microsoft Azure, or Google Cloud Platform.

Advantages

- Low initial investment.
- Automatic updates.
- Easy scalability.
- Global availability.
- Reduced maintenance.

Disadvantages

- Requires internet connectivity.
- Less direct control over hardware.
- Potential vendor dependency.

On-Premises Infrastructure

Infrastructure is installed and maintained within an organization's own data center.

Advantages

- Full control over hardware and security.
- Suitable for highly regulated industries.
- Customizable infrastructure.

Disadvantages

- High setup and maintenance costs.
- Requires dedicated IT staff.
- Limited scalability.
- Longer deployment times.

2. Cloud Service Models

Cloud services are categorized into three primary models based on the level of responsibility shared between the cloud provider and the customer.

Infrastructure as a Service (IaaS)

Definition

Infrastructure as a Service (IaaS) provides virtualized computing resources such as virtual machines, storage, networking, and operating systems over the internet.

The cloud provider manages the physical infrastructure, while the customer manages the operating system, applications, and data.

Examples

- Amazon EC2
- Microsoft Azure Virtual Machines
- Google Compute Engine

Suitable For

- Hosting web applications.
- Development and testing.
- Disaster recovery.
- Custom infrastructure requirements.

Platform as a Service (PaaS)

Definition

Platform as a Service (PaaS) provides a complete development platform, including operating systems, runtime environments, databases, and development tools.

Developers focus only on writing and deploying application code.

Examples

- Google App Engine
- Azure App Service
- AWS Elastic Beanstalk

Suitable For

- Web application development.
- API development.
- Rapid application deployment.

Software as a Service (SaaS)

Definition

Software as a Service (SaaS) delivers complete software applications over the internet.

Users simply access the application through a web browser without managing infrastructure or software installation.

Examples

- Gmail
- Microsoft 365
- Salesforce
- Dropbox

Suitable For

- Email services.
- Collaboration tools.
- Customer Relationship Management (CRM).
- Office productivity.

Comparison of Cloud Service Models

Feature	IaaS	PaaS	SaaS
Infrastructure Managed By	Customer	Cloud Provider	Cloud Provider
Operating System	Customer	Cloud Provider	Cloud Provider
Application Development	Customer	Customer	Not Required
Flexibility	High	Moderate	Low
User Responsibility	Highest	Medium	Lowest

3. Cloud Deployment Models

Cloud deployment models describe how cloud infrastructure is made available to users.

Public Cloud

Infrastructure is owned and managed by a third-party cloud provider and shared among multiple customers.

Examples

- AWS
- Azure
- Google Cloud Platform

Advantages

- Low cost.
- Easy scalability.
- No hardware maintenance.

Disadvantages

- Shared infrastructure.
- Less customization.

Private Cloud

Infrastructure is dedicated to a single organization.

Advantages

- Greater security.
- Better control.
- Regulatory compliance.

Disadvantages

- Higher cost.
- Organization manages infrastructure.

Hybrid Cloud

Combines Public and Private Cloud environments.

Organizations can keep sensitive workloads on a private cloud while using the public cloud for scalable resources.

Advantages

- Flexibility.
- Cost optimization.
- Improved disaster recovery.

Community Cloud

Infrastructure is shared by multiple organizations with similar security or regulatory requirements.

Examples

- Government organizations.
- Healthcare institutions.
- Educational institutions.

Comparison of Deployment Models

Deployment Model	Ownership	Best For
Public Cloud	Cloud Provider	General-purpose applications
Private Cloud	Single Organization	Sensitive workloads

Hybrid Cloud	Shared	Mixed workloads
Community Cloud	Multiple Organizations	Industry-specific collaboration

4. Cloud Service Providers

Several organizations provide cloud computing platforms and services.

Amazon Web Services (AWS)

AWS is the world's largest cloud platform, offering more than 200 cloud services.

Popular services include:

- EC2
- S3
- RDS
- Lambda
- VPC
- ECS
- DynamoDB

Strengths

- Largest market share.
- Extensive service portfolio.
- Global infrastructure.
- Strong DevOps ecosystem.

Microsoft Azure

Azure is Microsoft's cloud platform and integrates closely with Windows and Microsoft enterprise products.

Popular services include:

- Azure Virtual Machines
- Azure SQL Database
- Azure Functions
- Azure Blob Storage

Google Cloud Platform (GCP)

GCP is Google's cloud platform, known for its strengths in data analytics, artificial intelligence, and Kubernetes.

Popular services include:

- Compute Engine
- Cloud Storage
- BigQuery

- Cloud Functions
- Google Kubernetes Engine (GKE)

Comparison of Major Cloud Providers

AWS	Azure	GCP
Largest cloud provider	Strong Microsoft integration	Strong AI and Data Analytics
Extensive service offerings	Enterprise-focused	Kubernetes leadership
Excellent DevOps support	Hybrid cloud capabilities	Machine Learning services

5. Amazon EC2 (Elastic Compute Cloud)

What is Amazon EC2?

Amazon Elastic Compute Cloud (EC2) is a web service that provides scalable virtual servers, known as **instances**, in the AWS Cloud.

It allows users to quickly launch, manage, and terminate virtual machines without investing in physical hardware.

Benefits of EC2

- On-demand virtual servers.
- Flexible instance types.
- Easy scaling.
- Pay-as-you-go pricing.
- Integration with other AWS services.

EC2 Instances

An **EC2 Instance** is a virtual machine running in AWS.

Each instance includes:

- CPU
- Memory
- Storage
- Network interface
- Operating system

Launching an EC2 Instance (Console Walkthrough)

Typical steps to launch an EC2 instance:

1. Sign in to the AWS Management Console.
2. Open the **EC2 Dashboard**.
3. Click **Launch Instance**.
4. Enter an instance name.
5. Select an **Amazon Machine Image (AMI)**.

6. Choose an **Instance Type**.
7. Create or select a **Key Pair**.
8. Configure **Security Groups**.
9. Configure storage if needed.
10. Review settings and click **Launch Instance**.

The instance becomes available within a few minutes.

Amazon Machine Image (AMI)

An **Amazon Machine Image (AMI)** is a preconfigured template used to create EC2 instances.

An AMI contains:

- Operating system
- Application software
- Configuration settings
- Required dependencies

Types of AMIs

- AWS-provided AMIs
- Community AMIs
- Marketplace AMIs
- Custom AMIs

Custom AMIs can be created from existing EC2 instances for reuse.

EC2 Instance Types

AWS provides different instance families optimized for various workloads.

Examples:

- General Purpose
- Compute Optimized
- Memory Optimized
- Storage Optimized
- GPU Instances

t2.micro

t2.micro is a small General Purpose instance commonly included in the AWS Free Tier and is suitable for learning and small applications.

Security Groups

A **Security Group** acts as a virtual firewall for EC2 instances.

It controls:

- **Inbound Rules** – Traffic allowed into the instance.
- **Outbound Rules** – Traffic allowed out of the instance.

Example:

Port	Protocol	Purpose
22	SSH	Remote Linux access
80	HTTP	Web traffic
443	HTTPS	Secure web traffic

Key Pairs

A **Key Pair** is used for secure authentication when connecting to an EC2 instance.

It consists of:

- Public Key (stored by AWS)
- Private Key (.pem file, stored securely by the user)

Connecting to Linux EC2

```
ssh -i mykey.pem ec2-user@public-ip-address
```

The .pem file should be kept secure and never shared publicly.

6. Amazon ECS (Elastic Container Service)

What is a Container?

A **container** is a lightweight, portable package that includes an application and all its dependencies, ensuring consistent execution across different environments.

Containers are commonly created using **Docker**.

What is Amazon ECS?

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service provided by AWS.

It simplifies the deployment, management, and scaling of containerized applications.

ECS vs EC2

Amazon EC2	Amazon ECS
Provides virtual machines	Manages containerized applications
User manages the operating system	AWS manages container orchestration
Suitable for traditional applications	Suitable for Docker-based applications

Creating and Managing Containers in ECS

Typical workflow:

1. Build a Docker image.
2. Push the image to Amazon Elastic Container Registry (ECR) or another registry.
3. Create an ECS Cluster.
4. Define a **Task Definition** (container configuration).
5. Create an ECS Service.
6. Deploy and manage containers.
7. Monitor application health and scaling.

7. Amazon S3 (Simple Storage Service)

What is Amazon S3?

Amazon Simple Storage Service (Amazon S3) is a highly scalable, durable, and secure **object storage service** provided by AWS.

It is designed to store and retrieve any amount of data from anywhere over the internet.

Amazon S3 is commonly used for:

- File storage
- Website hosting
- Backup and recovery
- Data archiving
- Media storage
- Big Data and Analytics
- Application assets

Key Concepts

Bucket

A **Bucket** is a logical container used to store objects in Amazon S3.

- Bucket names must be globally unique.
- Buckets are created within a specific AWS Region.
- Each bucket can contain millions of objects.

Example:

student-documents
company-backups
project-images

Object

An **Object** is the actual file stored inside an S3 bucket.

An object consists of:

- File (data)
- Metadata
- Unique key (file name/path)

Example:

Bucket:
project-files

Objects:
report.pdf
logo.png
video.mp4

Creating an S3 Bucket (Console Walkthrough)

Typical steps:

1. Open the **AWS Management Console**.
2. Navigate to **Amazon S3**.
3. Click **Create Bucket**.
4. Enter a globally unique bucket name.
5. Select an AWS Region.
6. Configure permissions and settings.
7. Click **Create Bucket**.

The bucket becomes available immediately.

Uploading and Downloading Objects

Upload

1. Open the bucket.
2. Click **Upload**.
3. Select files.
4. Confirm upload.

Download

1. Select the required object.
2. Click **Download**.

Bucket Permissions

Permissions determine who can access bucket contents.

Private Bucket

- Default setting.
- Only authorized users can access data.

Public Bucket

- Accessible over the internet.
- Suitable for static website content or publicly available files.

Best Practice: Keep buckets private unless public access is explicitly required.

S3 Storage Classes

Storage classes allow users to optimize storage costs based on data access frequency.

1. S3 Standard

Designed for frequently accessed data.

Features:

- High durability.
- Low latency.
- High availability.

Use cases:

- Web applications.
- Frequently accessed files.
- Dynamic websites.

2. Intelligent-Tiering

Automatically moves objects between storage tiers based on access patterns.

Benefits:

- Reduces storage costs.
- No performance impact.
- No manual intervention.

Suitable for unpredictable workloads.

3. Standard-Infrequent Access (Standard-IA)

Designed for data accessed less frequently but requiring immediate retrieval.

Use cases:

- Backup files.
- Disaster recovery.

- Long-term project data.

4. Glacier

Designed for long-term archival.

Features:

- Very low storage cost.
- Retrieval takes minutes to hours.

Suitable for:

- Archived documents.
- Compliance records.
- Historical backups.

Comparison of S3 Storage Classes

Storage Class	Access Frequency	Retrieval Time	Typical Use Case
Standard	Frequent	Immediate	Active applications
Intelligent-Tiering	Variable	Immediate	Unpredictable access patterns
Standard-IA	Infrequent	Immediate	Backup and disaster recovery
Glacier	Rare	Minutes to hours	Long-term archive

Lifecycle Policies

A **Lifecycle Policy** automatically transitions or deletes objects based on predefined rules.

Example:

- Move files to Glacier after 90 days.
- Delete temporary files after one year.

Benefits:

- Reduces storage costs.
- Automates data lifecycle management.
- Minimizes manual administration.

Versioning

Versioning preserves multiple versions of an object.

Benefits:

- Recover accidentally deleted files.
- Restore previous versions.
- Protect against unintended modifications.

8. Amazon EBS (Elastic Block Store)

What is Amazon EBS?

Amazon Elastic Block Store (EBS) provides persistent **block-level storage** for Amazon EC2 instances.

Unlike the temporary storage of an EC2 instance, EBS volumes retain data even after the instance is stopped.

Why Use EBS?

EBS is ideal for:

- Operating system disks.
- Databases.
- Enterprise applications.
- File systems.
- Boot volumes.

Creating and Attaching an EBS Volume

Typical process:

1. Open the EC2 Dashboard.
2. Select **Volumes**.
3. Click **Create Volume**.
4. Choose size and type.
5. Create the volume.
6. Attach it to an EC2 instance.
7. Format and mount the volume inside the operating system.

EBS Volume Types

gp2 (General Purpose SSD)

- Balanced price and performance.
- Suitable for general workloads.

gp3 (General Purpose SSD)

- Improved performance over gp2.
- Lower cost.
- Independent performance configuration.

Recommended for most applications.

io1 (Provisioned IOPS SSD)

Designed for high-performance workloads.

Suitable for:

- Enterprise databases.
- Financial applications.
- High I/O systems.

EBS Snapshots

A **Snapshot** is a backup of an EBS volume stored in Amazon S3.

Benefits:

- Disaster recovery.
- Backup.
- Restore deleted volumes.
- Create new volumes quickly.

Snapshots are incremental, meaning only changed blocks are stored after the first snapshot.

Amazon S3 vs Amazon EBS

Amazon S3	Amazon EBS
Object Storage	Block Storage
Stores files as objects	Stores raw disk blocks
Accessible over the internet	Attached to EC2 instances
Highly scalable	Persistent storage for virtual machines
Suitable for backups, media, and documents	Suitable for operating systems and databases

9. Amazon VPC (Virtual Private Cloud)

What is Amazon VPC?

Amazon Virtual Private Cloud (VPC) is a logically isolated virtual network within AWS where cloud resources can be securely deployed.

It gives organizations full control over:

- IP addressing.
- Subnets.
- Routing.
- Security.
- Internet access.

Components of a VPC

A VPC typically includes:

- Subnets
- Route Tables
- Internet Gateway
- NAT Gateway
- Security Groups
- Network ACLs
- EC2 Instances

Public vs Private Subnet

Public Subnet

A subnet connected to the internet through an Internet Gateway.

Suitable for:

- Web servers.
- Load Balancers.
- Bastion Hosts.

Private Subnet

A subnet without direct internet access.

Suitable for:

- Databases.
- Internal application servers.
- Backend services.

Comparison

Public Subnet	Private Subnet
Internet accessible	No direct internet access
Hosts web applications	Hosts databases and internal services
Uses Internet Gateway	Uses NAT Gateway for outbound internet access

Route Tables

A **Route Table** determines how network traffic moves within a VPC.

Each route contains:

- Destination
- Target

Example:

Destination	Target
-------------	--------

Local VPC	Local
0.0.0.0/0	Internet Gateway

Internet Gateway (IGW)

An **Internet Gateway** enables communication between a VPC and the internet.

Characteristics:

- Attached to one VPC.
- Allows inbound and outbound internet traffic.
- Required for Public Subnets.

NAT Gateway

A **Network Address Translation (NAT) Gateway** enables instances in Private Subnets to access the internet **without allowing inbound internet connections**.

Common uses:

- Download software updates.
- Install packages.
- Access external APIs securely.

Security Groups

A **Security Group** acts as a virtual firewall for EC2 instances.

It controls:

- Inbound traffic.
- Outbound traffic.

Example rules:

Port Purpose

22 SSH

80 HTTP

443 HTTPS

VPC Peering

VPC Peering establishes a private connection between two VPCs.

Benefits:

- Secure communication.
- No internet required.
- Low network latency.

Common use cases:

- Connecting production and testing environments.
- Sharing internal services.
- Multi-region architectures.

10. Elastic Load Balancing (ELB)

What is Elastic Load Balancing?

Elastic Load Balancing (ELB) automatically distributes incoming application traffic across multiple targets such as EC2 instances or containers.

Benefits include:

- High availability.
- Improved fault tolerance.
- Better scalability.
- Efficient resource utilization.

Why Use Load Balancers?

Without a Load Balancer:

- One server receives all traffic.
- Increased risk of overload and downtime.

With a Load Balancer:

- Traffic is distributed evenly.
- Failed instances are automatically removed from service.
- Application availability improves.

Types of Load Balancers

AWS offers several load balancer types, with the most common being:

- **Application Load Balancer (ALB)**
- **Network Load Balancer (NLB)**

Application Load Balancer (ALB)

The **Application Load Balancer** operates at **Layer 7 (Application Layer)** of the OSI model.

It understands:

- HTTP

- HTTPS
- URL paths
- Hostnames
- HTTP headers

Features

- Path-based routing.
- Host-based routing.
- SSL termination.
- WebSocket support.
- Integration with ECS and Kubernetes.

Example

example.com/login



Authentication Service

example.com/products



Product Service

The ALB routes requests based on the URL path.

Network Load Balancer (NLB)

The **Network Load Balancer** operates at **Layer 4 (Transport Layer)**.

It routes traffic using:

- TCP
- UDP
- TLS

Features

- Extremely high performance.
- Ultra-low latency.
- Static IP support.
- Millions of requests per second.

Suitable for:

- Gaming servers.
- Financial systems.
- IoT platforms.
- Real-time applications.

ALB vs NLB

Application Load Balancer (ALB)	Network Load Balancer (NLB)
Layer 7	Layer 4
HTTP/HTTPS	TCP/UDP/TLS
Path-based routing	IP and Port routing
Content-aware routing	Very high performance
Best for web applications	Best for low-latency applications

Target Groups

A **Target Group** is a collection of backend resources that receive traffic from a Load Balancer.

Targets may include:

- EC2 Instances
- ECS Tasks
- IP Addresses
- Lambda Functions

Health Checks

Load Balancers continuously monitor backend targets using **Health Checks**.

If a target becomes unhealthy:

- It stops receiving traffic.
- Requests are routed to healthy targets.
- When the target recovers, it is automatically added back.

This ensures continuous application availability.

11. Amazon RDS (Relational Database Service)

What is Amazon RDS?

Amazon Relational Database Service (Amazon RDS) is a fully managed database service that simplifies the setup, operation, scaling, and maintenance of relational databases in the AWS Cloud.

Instead of installing and managing database software manually, AWS handles tasks such as:

- Software installation
- Database patching
- Automatic backups
- Monitoring
- Scaling
- High availability

This allows developers to focus on application development rather than database administration.

Benefits of Amazon RDS

- Fully managed database service.
- Automatic software updates.
- Automated backups.
- Easy scalability.
- Built-in monitoring.
- High availability.
- Improved security.

Managed vs Self-Managed Databases

Managed Database (RDS)	Self-Managed Database
Managed by AWS	Managed by the organization
Automatic backups	Manual backups
Automatic patching	Manual updates
Easy scaling	Manual infrastructure upgrades
Built-in monitoring	Custom monitoring required

Creating an RDS Database (Console Walkthrough)

Typical steps:

1. Open the **AWS Management Console**.
2. Navigate to **Amazon RDS**.
3. Click **Create Database**.
4. Select the database engine.
5. Configure the instance size.
6. Set the database username and password.
7. Configure storage and networking.
8. Review and create the database.

The database becomes available after provisioning.

Supported Database Engines

Amazon RDS supports several popular relational databases, including:

- **MySQL**
- **PostgreSQL**
- **MariaDB**
- **Oracle**
- **Microsoft SQL Server**
- **Amazon Aurora**

Amazon Aurora

Amazon Aurora is an AWS-designed relational database engine compatible with MySQL and PostgreSQL.

Advantages

- Higher performance.
- Automatic replication.
- Faster failover.
- Better scalability.
- High availability.

Multi-AZ Deployment

Multi-AZ (Availability Zone) deployment improves database availability by maintaining a synchronized standby replica in another Availability Zone.

Benefits

- Automatic failover during hardware or Availability Zone failures.
- High availability.
- Increased fault tolerance.
- Minimal downtime.

Automated Backups

Amazon RDS automatically performs database backups based on the configured backup retention period.

Backup Features

- Point-in-time recovery.
- Automatic daily snapshots.
- Transaction log backups.
- Easy restoration.

Snapshots

A **Snapshot** is a manual backup of an RDS database.

Snapshots are commonly used for:

- Long-term backup.
- Migration.
- Disaster recovery.
- Database cloning.

12. Amazon DynamoDB

What is DynamoDB?

Amazon DynamoDB is a fully managed **NoSQL database service** that delivers high performance, low latency, and automatic scaling.

Unlike relational databases, DynamoDB stores data as **key-value pairs** or **documents**.

Features

- Fully managed.
- Serverless.
- Automatic scaling.
- Single-digit millisecond response time.
- High availability.
- Built-in security.

NoSQL Database Concepts

A NoSQL database does not use traditional tables and relationships like relational databases.

Instead, data is stored as:

- Key-value pairs.
- Documents.
- Wide-column data.
- Graph structures (depending on the database type).

DynamoDB primarily supports:

- Key-value storage.
- Document storage.

Amazon RDS vs DynamoDB

Amazon RDS	Amazon DynamoDB
Relational (SQL)	NoSQL
Tables with rows and columns	Key-value and document model
Fixed schema	Flexible schema
SQL queries	API-based operations
Suitable for complex relationships	Suitable for high-speed scalable applications

When to Use RDS

Choose Amazon RDS when:

- Relationships between tables are important.
- SQL queries are required.
- Transactions are complex.
- Applications use structured data.

Examples:

- Banking systems.
- ERP applications.
- Inventory management.

When to Use DynamoDB

Choose DynamoDB when:

- High scalability is required.
- Very low latency is needed.
- Data structure changes frequently.
- Millions of requests must be handled efficiently.

Examples:

- Gaming leaderboards.
- Shopping carts.
- IoT applications.
- Social media feeds.

Creating a DynamoDB Table

Typical steps:

1. Open the **AWS Management Console**.
2. Navigate to **Amazon DynamoDB**.
3. Click **Create Table**.
4. Enter the table name.
5. Define the primary key.
6. Configure capacity settings.
7. Create the table.

Primary Keys

Every DynamoDB table requires a primary key.

There are two options:

Partition Key

A single unique key used to identify each item.

Example:

StudentID
EmployeeID
OrderID

Composite Primary Key

Consists of:

- Partition Key
- Sort Key

The Partition Key identifies a group of related items, while the Sort Key uniquely identifies items within that group.

Example:

Partition Key	Sort Key
StudentID	Semester
CustomerID	OrderDate

Query vs Scan

Query

- Searches using the primary key.
- Fast.
- Efficient.
- Recommended.

Scan

- Reads every item in the table.
- Slower.
- Consumes more resources.
- Suitable only for small datasets or administrative tasks.

High-Scale Low-Latency Applications

DynamoDB is ideal for applications requiring:

- Millions of users.
- High throughput.
- Low response times.
- Automatic scaling.
- Global availability.

13. AWS Lambda

What is Serverless Computing?

Serverless Computing is a cloud computing model in which developers write and deploy code without managing servers.

The cloud provider automatically handles:

- Infrastructure provisioning.
- Scaling.
- Maintenance.
- Availability.

Developers pay only for actual execution time.

What is AWS Lambda?

AWS Lambda is AWS's serverless compute service that executes code automatically in response to events.

No server management is required.

Lambda Features

- Serverless.
- Automatic scaling.
- Event-driven execution.
- Supports multiple programming languages.
- Pay-per-use pricing.
- High availability.

Event-Driven Execution

A Lambda function runs only when triggered by an event.

Common triggers include:

- File upload.
- API request.
- Database update.
- Scheduled event.
- Message queue.

Common Event Sources

Amazon S3

Trigger a Lambda function whenever:

- A file is uploaded.
- A file is deleted.

Example:

Upload image → Automatically generate thumbnail.

API Gateway

Trigger Lambda whenever an HTTP request is received.

Example:

GET /students

POST /login

PUT /profile

DynamoDB Streams

Trigger Lambda whenever data in a DynamoDB table changes.

Example:

- New record inserted.
- Record updated.
- Record deleted.

Creating a Lambda Function (Console Walkthrough)

Typical steps:

1. Open AWS Lambda.
2. Click **Create Function**.
3. Enter a function name.
4. Select a runtime.
5. Write or upload the code.
6. Configure permissions.
7. Deploy the function.
8. Test using sample events.

Lambda Runtime

AWS Lambda supports multiple programming languages.

Common runtimes include:

- Python
- Java
- Node.js
- C#
- Go
- Ruby

Pricing Model

Lambda uses a **pay-per-execution** model.

Charges are based on:

- Number of requests.
- Execution duration.
- Memory allocated.

No charges apply while the function is idle.

Integrating Lambda with AWS Services

Lambda integrates with many AWS services, including:

- Amazon S3
- API Gateway
- DynamoDB
- SNS
- SQS
- CloudWatch
- EventBridge

This enables highly automated, event-driven workflows.

14. AWS API Gateway

What is AWS API Gateway?

Amazon API Gateway is a fully managed service that allows developers to create, publish, secure, monitor, and manage APIs.

It acts as the entry point for applications and backend services.

Benefits

- Fully managed.
- Easy API creation.
- Scalable.
- Secure.
- Integrates with Lambda.
- Supports monitoring and logging.

Creating a REST API (Console Walkthrough)

Typical steps:

1. Open API Gateway.
2. Create a new REST API.
3. Define resources.
4. Configure HTTP methods.
5. Integrate backend services.
6. Deploy the API.
7. Test the endpoints.

HTTP Methods

Common HTTP methods include:

Method	Purpose
GET	Retrieve data
POST	Create data
PUT	Update data
DELETE	Delete data
PATCH	Partially update data

Routes

Routes define the API endpoints.

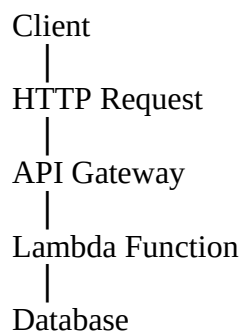
Example:

/students
/students/{id}
/courses
/login

Each route can support multiple HTTP methods.

API Gateway + Lambda Integration

One of the most common serverless architectures is:



Example Workflow:

1. User sends an HTTP request.
2. API Gateway receives the request.
3. API Gateway invokes a Lambda function.
4. Lambda processes the request.
5. Lambda accesses a database if needed.
6. Response is returned to the client.

Deployment Stages

API Gateway supports multiple deployment stages.

Common stages include:

Stage	Purpose
Development (Dev)	Developer testing
Staging	Pre-production testing
Production (Prod)	Live application

Using stages helps isolate environments and supports safe application releases.

Throttling

Throttling limits the number of API requests that clients can make within a specified period.

Benefits include:

- Preventing server overload.
- Protecting backend services.
- Improving application stability.

Basic API Security

Common security mechanisms include:

- API Keys.
- IAM Authentication.
- Amazon Cognito.
- OAuth.
- HTTPS encryption.
- Rate limiting.

These features help protect APIs from unauthorized access and abuse.

Best Practices

- Choose the appropriate cloud service model (IaaS, PaaS, or SaaS) based on application requirements.
- Use **Public Cloud** for scalable, general-purpose workloads and **Private/Hybrid Cloud** for sensitive applications.
- Select EC2 instance types according to workload characteristics to optimize performance and cost.
- Apply the principle of **least privilege** when configuring Security Groups by opening only the required ports.
- Store .pem key files securely and never commit them to version control systems.
- Prefer **Amazon ECS** for deploying and managing containerized applications instead of manually managing Docker containers on EC2 instances.
- Monitor resource utilization regularly to avoid unnecessary cloud costs.

- Take advantage of cloud elasticity by scaling resources based on demand.
- Organize S3 buckets by application or environment and enable **Versioning** for critical data.
- Use **Lifecycle Policies** to automatically transition infrequently accessed data to lower-cost storage classes like Glacier.
- Choose **gp3** as the default EBS volume type for most workloads due to its cost and performance benefits.
- Create regular **EBS Snapshots** to support backup and disaster recovery.
- Design VPCs with **Public Subnets** for internet-facing resources and **Private Subnets** for databases and internal services.
- Restrict Security Group rules to only the necessary ports and trusted IP addresses.
- Use **Application Load Balancers (ALB)** for HTTP/HTTPS web applications requiring content-based routing.
- Use **Network Load Balancers (NLB)** for high-performance TCP/UDP workloads requiring ultra-low latency.
- Configure **Health Checks** to ensure traffic is routed only to healthy application instances.
- Use **Amazon RDS** for applications requiring relational data, SQL queries, and complex transactions.
- Enable **Multi-AZ Deployments** and automated backups for production RDS databases to improve availability and disaster recovery.
- Choose **Amazon DynamoDB** for high-scale, low-latency applications with flexible data models.
- Design DynamoDB tables using efficient **Partition Keys** to optimize query performance.
- Prefer **Query** operations over **Scan** to minimize latency and reduce resource consumption.
- Build event-driven applications using **AWS Lambda** to eliminate server management and reduce operational overhead.
- Configure Lambda functions with the **minimum required permissions** using AWS Identity and Access Management (IAM).
- Use **AWS API Gateway** as the secure entry point for REST APIs and integrate it with Lambda for serverless backends.
- Separate environments using API Gateway deployment stages such as **Development**, **Staging**, and **Production**.
- Enable throttling, authentication, and HTTPS to improve API security and reliability.

Quick Revision Points

- **Cloud Computing** delivers computing resources over the internet on a pay-as-you-go basis.
- **Virtualization** enables multiple virtual machines to run on a single physical server using a hypervisor.
- **Service-Oriented Architecture (SOA)** organizes applications into reusable, loosely coupled services.
- Cloud Service Models include **IaaS**, **PaaS**, and **SaaS**, offering increasing levels of abstraction and management by the cloud provider.
- Cloud Deployment Models are **Public**, **Private**, **Hybrid**, and **Community Cloud**, each suited to different organizational needs.

- Major cloud providers include **Amazon Web Services (AWS)**, **Microsoft Azure**, and **Google Cloud Platform (GCP)**.
- **Amazon EC2** provides scalable virtual servers called **instances**, which are launched from **Amazon Machine Images (AMIs)** and protected by **Security Groups** and **Key Pairs**.
- **Amazon ECS (Elastic Container Service)** is a fully managed AWS service for deploying, managing, and scaling Docker-based containerized applications.
- **Amazon S3** is AWS's scalable object storage service that stores data as **Objects** inside **Buckets**.
- Common S3 storage classes include **Standard**, **Intelligent-Tiering**, **Standard-IA**, and **Glacier**, each optimized for different access patterns and costs.
- **Lifecycle Policies** automate data transitions and deletion, while **Versioning** protects against accidental deletion or modification.
- **Amazon EBS (Elastic Block Store)** provides persistent block storage for EC2 instances and supports volume types such as **gp2**, **gp3**, and **io1**.
- **EBS Snapshots** provide incremental backups for disaster recovery and volume restoration.
- **Amazon VPC** is a logically isolated virtual network containing components such as **Subnets**, **Route Tables**, **Internet Gateways**, **NAT Gateways**, and **Security Groups**.
- **Public Subnets** provide internet access through an Internet Gateway, whereas **Private Subnets** use a NAT Gateway for secure outbound internet connectivity.
- **VPC Peering** enables private communication between two VPCs without traversing the public internet.
- **Elastic Load Balancing (ELB)** distributes incoming traffic across multiple targets to improve availability and scalability.
- **Application Load Balancer (ALB)** operates at **Layer 7** and supports HTTP/HTTPS with content-based routing, while **Network Load Balancer (NLB)** operates at **Layer 4** and is optimized for high-performance TCP/UDP traffic.
- **Amazon RDS** is a fully managed relational database service supporting engines such as **MySQL**, **PostgreSQL**, **SQL Server**, and **Amazon Aurora**.
- **Multi-AZ Deployments** improve database availability by maintaining a synchronized standby replica for automatic failover.
- **Amazon DynamoDB** is a fully managed NoSQL database optimized for high scalability, low latency, and automatic scaling.
- DynamoDB uses **Partition Keys** and optional **Sort Keys** as primary keys, and **Query** operations are more efficient than **Scan** operations.
- **AWS Lambda** is a serverless compute service that executes code in response to events without requiring server management.
- Lambda commonly integrates with **Amazon S3**, **API Gateway**, and **DynamoDB Streams** to build event-driven applications.
- **AWS API Gateway** enables developers to create, secure, deploy, and manage REST APIs while integrating seamlessly with backend services such as Lambda.
- API Gateway supports standard HTTP methods (**GET**, **POST**, **PUT**, **DELETE**, **PATCH**), deployment stages (**Development**, **Staging**, **Production**), and security features such as authentication, throttling, and HTTPS encryption.

Complete Summary

Cloud Computing enables organizations to access computing resources over the internet with scalability, flexibility, and a pay-as-you-go pricing model. AWS provides a comprehensive

ecosystem of cloud services that support infrastructure, storage, networking, databases, containers, serverless computing, and API management.

The key AWS services covered in this module include:

AWS Service	Primary Purpose
EC2	Virtual servers in the cloud
ECS	Container orchestration and management
S3	Scalable object storage
EBS	Persistent block storage for EC2
VPC	Isolated virtual networking
Elastic Load Balancer (ALB/NLB)	Traffic distribution across application resources
RDS	Managed relational databases
DynamoDB	Fully managed NoSQL database
Lambda	Serverless event-driven computing
API Gateway	Creation, deployment, and management of REST APIs

Together, these services form the foundation of modern cloud-native application development and are widely used in DevOps, scalable web applications, microservices architectures, and enterprise cloud solutions.